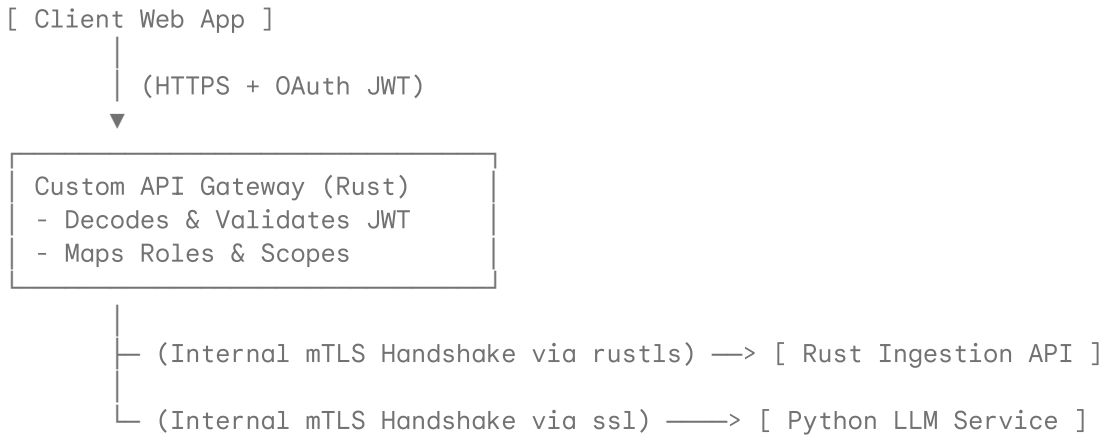
Project Millipede: Zero-Trust Security Specification

This document maps out the end-to-end security design for `millipede`, focusing on our custom Rust Gateway and internal Mutual TLS (mTLS) network.

🔐 The Security Boundary Topology

We reject the traditional "hard shell, soft underbelly" layout. Instead, we implement a **Zero-Trust Architecture (ZTA)** that extends past the API gateway and into each microservice node.



1. Edge Authentication: JWT Validation

The client dashboard authenticates against an external identity provider to receive a JWT. The Custom Rust Gateway decodes the signature, verifies expiration timestamps, checks user roles (`manager`), and strips external claims.

2. Service-to-Service Security: Mutual TLS (mTLS)

Downstream internal microservices do not trust the API Gateway by default. Every inter-service network call requires a Mutual TLS connection:

- **Rust Nodes (`rustls`):** Compile-time safe, modern, and highly optimized TLS implementation in Rust.
- **Python Nodes (`ssl` / `pyOpenSSL`):** The Python LLM service validates incoming client certificates before accepting socket requests.

🛠️ Bootstrapping the Local Certificate Authority (CA)

To bypass complex enterprise PKI setups during local development, we use a simple automation script inside `/infra` to generate self-signed certificates:

```
# 1. Generate Root Private Key and self-signed certificate authority
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 1024 -out rootCA.cr

# 2. Generate Gateway Private Key and Certificate Signing Request (CSR)
openssl genrsa -out gateway.key 2048
openssl req -new -key gateway.key -out gateway.csr
```

```
# 3. Sign the Gateway Certificate using our Root CA
openssl x509 -req -in gateway.csr -CA rootCA.crt -CAkey rootCA.key \
  -CAcreateserial -out gateway.crt -days 500 -sha256

# 4. Generate & sign internal microservice keys
openssl genrsa -out service.key 2048
openssl req -new -key service.key -out service.csr
openssl x509 -req -in service.csr -CA rootCA.crt -CAkey rootCA.key \
  -CAcreateserial -out service.crt -days 500 -sha256
```

These generated certificates are mounted directly into our Docker containers, establishing a secure network boundary.